EXERCÍCIOS PRÁTICOS

Tradução dirigida por sintaxe, AST, tabela de símbolos e escopos

Desenvolvimento do compilador MINIC em C e Python

Atividades baseadas nas Aulas 13 e 14.

Orientações gerais

Este caderno propõe dez exercícios. Em cada atividade, implemente o componente solicitado nas duas linguagens: C e Python. Os exemplos de entrada são pequenos programas-fonte com extensão .c; as saídas indicadas são as saídas esperadas da ferramenta/analisador desenvolvido, e não a execução dos programas-fonte como aplicações independentes.

Escopo dos conteúdos: atributos sintetizados e herdados, árvores e grafos de dependência, ordem topológica, SDDs S-atribuídas/L-atribuídas, ações semânticas, construção incremental da AST, tabela de símbolos, escopo léxico, pilha de escopos, resolução de identificadores, sombreamento, duplicidade no escopo corrente e diagnósticos de nomes. Não se pressupõem geração de código, otimização, fluxo de controle ou análise de tipos além da propagação de int/real abordada na atividade.

Convenções dos casos de teste

Cada caso mostra o conteúdo de um arquivo-fonte .c e a linha de saída exata a ser produzida pelo programa do exercício. Preserve a ordem de visita indicada em cada enunciado. Os exemplos usam apenas o subconjunto necessário ao conceito testado.

Materiais de referência

Aula 13 — Tradução dirigida por sintaxe;

Aula 14 — Tabela de símbolos e escopos.

Os exercícios foram delimitados ao conteúdo textual desses dois arquivos anexados.

Exercício 1 — Atributos sintetizados: avaliação de expressões

Objetivo

Implemente a avaliação de expressões aritméticas por atributos sintetizados. O valor de cada nó interno deve ser calculado a partir dos valores dos filhos e propagado até a raiz.

Implementação em C e Python

Em C e em Python, represente valores léxicos nas folhas e calcule +, -, * e / inteiro. Respeite precedência (* e / antes de + e -), parênteses e associatividade à esquerda. A interface deve receber um programa-fonte MINIC e emitir o valor do inicializador de r.

Critérios e limites

A implementação deve demonstrar que o atributo nasce nas folhas e sobe pela árvore. Considere entradas válidas e divisão inteira positiva; tratamento de divisão por zero não é exigido neste exercício.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste1_1.c

```
int r = 2 + 3 * 4;
```

Saída esperada: `valor(r)=14`

Caso 2 — arquivo teste1_2.c

```
int r = (2 + 3) * 4;
```

Saída esperada: `valor(r)=20`

Caso 3 — arquivo teste1_3.c

```
int r = 20 - 6 - 3;
```

Saída esperada: `valor(r)=11`

Caso 4 — arquivo teste1_4.c

```
int r = 18 / 3 - 2;
```

Saída esperada: `valor(r)=4`

Caso 5 — arquivo teste1_5.c

```
int r = 7 + 8 / 2 * 3;
```

Saída esperada: `valor(r)=19`

Exercício 2 — Construção incremental da AST

Objetivo

Construa uma AST para expressões aritméticas durante a análise, preservando a estrutura, a precedência, os parênteses e a associatividade.

Implementação em C e Python

Implemente em C e Python os nós Leaf/Identifier e Binary (ou estruturas equivalentes). A cada redução ou visita, crie o nó correspondente usando os nós já construídos. Produza a AST em notação prefixa: operadores como +(esq,dir), números como 2 e identificadores como id(a).

Critérios e limites

A árvore deve refletir a gramática, não apenas repetir o texto de entrada. Uma expressão como 20-6-3 deve ser agrupada como (20-6)-3.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste2_1.c

```
int r = 2 + 3 * 4;
```

Saída esperada: `AST(r)=+(2,*(3,4))`

Caso 2 — arquivo teste2_2.c

```
int r = (2 + 3) * 4;
```

Saída esperada: `AST(r)=*(+(2,3),4)`

Caso 3 — arquivo teste2_3.c

```
int r = 20 - 6 - 3;
```

Saída esperada: `AST(r)=-(-(20,6),3)`

Caso 4 — arquivo teste2_4.c

```
int r = a + b * c;
```

Saída esperada: `AST(r)=+(id(a),*(id(b),id(c)))`

Caso 5 — arquivo teste2_5.c

```
int r = (a + b) * (c - 2);
```

Saída esperada: `AST(r)=*(+(id(a),id(b)),-(id(c),2))`

Exercício 3 — Grafo de dependências e ordem de avaliação

Objetivo

A partir da árvore de expressão, construa o grafo de dependências dos atributos sintetizados e verifique uma ordem topológica de avaliação.

Implementação em C e Python

Implemente em C e Python a criação dos nós da expressão e das arestas filho→pai. Calcule uma ordem topológica e avalie a raiz. Para estes testes, emita exatamente o resumo: número de nós (folhas + operadores), número de arestas, se o grafo é acíclico e valor da raiz.

Critérios e limites

Uma árvore de expressão com n nós tem n−1 arestas. A ordem de avaliação deve colocar cada operando antes do operador que depende dele; o resumo solicitado facilita comparar as duas versões.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste3_1.c

```
int r = 2 + 3 * 4;
```

Saída esperada: `nos=5;arestas=4;acyclico=sim;valor_raiz=14`

Caso 2 — arquivo teste3_2.c

```
int r = (2 + 3) * 4;
```

Saída esperada: `nos=5;arestas=4;acyclico=sim;valor_raiz=20`

Caso 3 — arquivo teste3_3.c

```
int r = 20 - 6 - 3;
```

Saída esperada: `nos=5;arestas=4;acyclico=sim;valor_raiz=11`

Caso 4 — arquivo teste3_4.c

```
int r = 18 / 3 - 2;
```

Saída esperada: `nos=5;arestas=4;acyclico=sim;valor_raiz=4`

Caso 5 — arquivo teste3_5.c

```
int r = 7 + 8 / 2 * 3;
```

Saída esperada: `nos=7;arestas=6;acyclico=sim;valor_raiz=19`

Exercício 4 — Atributo herdado para listas de declarações

Objetivo

Implemente uma definição dirigida por sintaxe L-atribuída para propagar o tipo de uma declaração aos identificadores de uma lista.

Implementação em C e Python

Em C e Python, reconheça declarações do subconjunto int id-lista; e real id-lista;. Propague o tipo do símbolo de tipo para a lista e registre o tipo em cada identificador. Emita os pares lexema:tipo na ordem de declaração.

Critérios e limites

O atributo de tipo é contexto herdado pela lista; cada identificador recebe esse tipo. Não inclua conversões, inferência de tipos ou outros tipos além de int e real.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste4_1.c

```
int a;
```

Saída esperada: `simbolos=a:int`

Caso 2 — arquivo teste4_2.c

```
int a, b;
```

Saída esperada: `simbolos=a:int,b:int`

Caso 3 — arquivo teste4_3.c

```
real x, y, z;
```

Saída esperada: `simbolos=x:real,y:real,z:real`

Caso 4 — arquivo teste4_4.c

```
int a, b, c, d;
```

Saída esperada: `simbolos=a:int,b:int,c:int,d:int`

Caso 5 — arquivo teste4_5.c

```
real medida;
```

Saída esperada: `simbolos=medida:real`

Exercício 5 — Ações semânticas e ordem de construção

Objetivo

Associe ações semânticas pequenas à análise de expressões e registre a sequência em que folhas e nós Binary são criados.

Implementação em C e Python

Implemente em C e Python ações que criem folhas para números e identificadores e um nó Binary após seus operandos estarem disponíveis. Emita os eventos em pós-ordem, usando folha(valor), bin(operador) e, ao final, raiz=operador.

Critérios e limites

A sequência deve respeitar as dependências e a associatividade determinada pela árvore. Separar criação da AST e avaliação do valor é permitido, mas o evento registrado tem de corresponder à construção real.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste5_1.c

```
int r = 2 + 3 * 4;
```

Saída esperada: `eventos=folha(2),folha(3),folha(4),bin(*),bin(+);raiz=+`

Caso 2 — arquivo teste5_2.c

```
int r = (2 + 3) * 4;
```

Saída esperada: `eventos=folha(2),folha(3),bin(+),folha(4),bin(*);raiz=*`

Caso 3 — arquivo teste5_3.c

```
int r = 20 - 6 - 3;
```

Saída esperada: `eventos=folha(20),folha(6),bin(-),folha(3),bin(-);raiz=-`

Caso 4 — arquivo teste5_4.c

```
int r = 18 / 3 - 2;
```

Saída esperada: `eventos=folha(18),folha(3),bin(/),folha(2),bin(-);raiz=-`

Caso 5 — arquivo teste5_5.c

```
int r = 7 + 8 / 2 * 3;
```

Saída esperada: `eventos=folha(7),folha(8),folha(2),bin(/),folha(3),bin(*),bin(+);raiz=+`

Exercício 6 — Pilha de escopos e entrada/saída de blocos

Objetivo

Implemente a gestão do ambiente léxico como uma pilha de tabelas: abrir bloco cria um escopo e fechar bloco restaura o ambiente envolvente.

Implementação em C e Python

Em C e em Python, mantenha um escopo global G e crie escopos B1, B2 etc. em ordem de abertura. Ao fechar cada bloco, remova seu escopo do topo. Para cada teste, emita quantidade de escopos criados, quantidade ainda aberta e topo final.

Critérios e limites

Escopo é uma região; tabela é o armazenamento de declarações; ambiente é o conjunto de tabelas visíveis. Os cinco testes terminam com todos os blocos fechados.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste6_1.c

```
int x;
```

Saída esperada: `escopos_criados=0;abertos=0;topo=G`

Caso 2 — arquivo teste6_2.c

```
{
  int x;
}
```

Saída esperada: `escopos_criados=1;abertos=0;topo=G`

Caso 3 — arquivo teste6_3.c

```
{
  {
    int x;
  }
}
```

Saída esperada: `escopos_criados=2;abertos=0;topo=G`

Caso 4 — arquivo teste6_4.c

```
{
  int x;
  {
    real y;
  }
}
```

Saída esperada: `escopos_criados=2;abertos=0;topo=G`

Caso 5 — arquivo teste6_5.c

```
{
  {
    {
      int z;
    }
  }
}
```

Saída esperada: `escopos_criados=3;abertos=0;topo=G`

Exercício 7 — Resolução léxica e sombreamento

Objetivo

Resolva cada uso de identificador buscando do escopo mais interno até o global. Permita que declaração interna sombreie a externa e restaure o vínculo quando o bloco fechar.

Implementação em C e Python

Implemente em C e Python declarações int/real, blocos e atribuições simples. Registre para cada uso o lexema, o escopo da declaração visível e o tipo. Nomeie os escopos por ordem de abertura (G, B1, B2...). Emita os vínculos na ordem de ocorrência.

Critérios e limites

A busca deve parar na primeira declaração encontrada do escopo atual para fora. O mesmo lexema pode resolver para símbolos diferentes em pontos distintos do programa.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste7_1.c

```
int x;
x = 1;
```

Saída esperada: `vinculos=x@G:int`

Caso 2 — arquivo teste7_2.c

```
int x;
{
  real x;
  x = 2;
}
x = 3;
```

Saída esperada: `vinculos=x@B1:real,x@G:int`

Caso 3 — arquivo teste7_3.c

```
int x;
{
  x = 2;
}
```

Saída esperada: `vinculos=x@G:int`

Caso 4 — arquivo teste7_4.c

```
{
  int x;
  x = 1;
}
```

Saída esperada: `vinculos=x@B1:int`

Caso 5 — arquivo teste7_5.c

```
int x;
{
  {
    real x;
    x = 2;
  }
  x = 3;
}
```

Saída esperada: `vinculos=x@B2:real,x@G:int`

Exercício 8 — Duplicidade no escopo atual

Objetivo

Implemente a operação declarar, rejeitando nomes repetidos na tabela do escopo atual sem rejeitar automaticamente uma declaração que apenas sombreia outra externa.

Implementação em C e Python

Em C e Python, faça a busca de duplicidade somente na tabela corrente antes de inserir. Ao encontrar conflito, informe lexema, escopo e linhas da declaração repetida e da declaração original. Para este exercício, considere a primeira duplicidade como erro final.

Critérios e limites

A política de sombreamento pertence à especificação da linguagem; neste exercício, o sombreamento entre blocos é permitido, mas duplicidade no mesmo bloco é erro.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste8_1.c

```
int x;
int x;
```

Saída esperada: `erro=declaracao_duplicada;lexema=x;escopo=G;linha=2;conflito_linha=1`

Caso 2 — arquivo teste8_2.c

```
int x;
{
  int x;
}
```

Saída esperada: `resultado=OK`

Caso 3 — arquivo teste8_3.c

```
{
  real y;
  real y;
}
```

Saída esperada: `erro=declaracao_duplicada;lexema=y;escopo=B1;linha=3;conflito_linha=2`

Caso 4 — arquivo teste8_4.c

```
{
  int a;
  {
    int a;
  }
}
```

Saída esperada: `resultado=OK`

Caso 5 — arquivo teste8_5.c

```
int x;
{
  int x;
  int x;
}
```

Saída esperada: `erro=declaracao_duplicada;lexema=x;escopo=B1;linha=4;conflito_linha=3`

Exercício 9 — Diagnóstico de identificador não declarado

Objetivo

Use a tabela de símbolos para detectar usos sem declaração visível e emitir um diagnóstico acionável com categoria, lexema e localização.

Implementação em C e Python

Implemente em C e Python a resolução de nomes em atribuições simples e blocos. Se não houver declaração visível, imprima erro=identificador_nao_declarado, o lexema e a linha do uso. Caso todos os usos sejam resolvidos, emita resultado=OK.

Critérios e limites

A busca inclui escopos envolventes, mas não escopos que já foram fechados. Informe a primeira ocorrência não resolvida na ordem de visita ao programa.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste9_1.c

```
int x;
x = 1;
```

Saída esperada: `resultado=OK`

Caso 2 — arquivo teste9_2.c

```
z = 2;
```

Saída esperada: `erro=identificador_nao_declarado;lexema=z;linha=1`

Caso 3 — arquivo teste9_3.c

```
int x;
y = x;
```

Saída esperada: `erro=identificador_nao_declarado;lexema=y;linha=2`

Caso 4 — arquivo teste9_4.c

```
int x;
{
  x = 1;
}
```

Saída esperada: `resultado=OK`

Caso 5 — arquivo teste9_5.c

```
{
  int y;
  y = 1;
}
y = 2;
```

Saída esperada: `erro=identificador_nao_declarado;lexema=y;linha=5`

Exercício 10 — Vinculação de ocorrências da AST a símbolos

Objetivo

Durante a visita semântica da AST, associe cada nó de identificador à declaração visível correta. Preserve uma referência estável ao símbolo, não apenas o lexema.

Implementação em C e Python

Implemente em C e Python registros de símbolo com ID crescente pela ordem de declaração (#1, #2...). Cada nó de uso resolvido deve guardar/referenciar o ID do símbolo. Emita declarações e vínculos conforme os exemplos; em atribuições, visite primeiro o identificador à esquerda e depois os identificadores à direita.

Critérios e limites

Duas ocorrências com o mesmo lexema podem apontar para IDs diferentes por causa de sombreamento. Usos dentro de um bloco podem apontar para uma declaração externa visível.

Casos de teste — entrada .c e saída esperada

Caso 1 — arquivo teste10_1.c

```
int x;
x = 1;
```

Saída esperada: `declaracoes=#1:x@G;vinculos=x->#1`

Caso 2 — arquivo teste10_2.c

```
int x;
{
  real x;
  x = 2;
}
x = 3;
```

Saída esperada: `declaracoes=#1:x@G,#2:x@B1;vinculos=x@B1->#2,x@G->#1`

Caso 3 — arquivo teste10_3.c

```
int x;
{
  x = 2;
}
```

Saída esperada: `declaracoes=#1:x@G;vinculos=x@B1->#1`

Caso 4 — arquivo teste10_4.c

```
{
  int y;
  y = 1;
}
```

Saída esperada: `declaracoes=#1:y@B1;vinculos=y@B1->#1`

Caso 5 — arquivo teste10_5.c

```
int x;
{
  int y;
  y = x;
}
```

Saída esperada: `declaracoes=#1:x@G,#2:y@B1;vinculos=y@B1->#2,x@B1->#1`